

FAST-NUCES

FAST National University of Computer and Emerging Sciences

Red-Track-StudentHub

Project Proposal

Database Management Systems — Semester Project

Submitted To	Mr. Talha Shahid
Semester	4th Semester — Spring 2026
Department	Computer Science
University	FAST National University of Computer and Emerging Sciences
Submission Date	May 2026

Group Members

#	Name	Roll Number
1	Mushaf Ali Mir	24P-0582
2	Adnan Osama	24K-0598
3	Zaid Bin Zubair	24K-0796

1. Introduction

Red-Track-StudentHub is a comprehensive web-based academic portal designed for university students. The system consolidates three critical academic needs into a single platform: teacher performance reviews, past examination paper retrieval, and course management — all backed by a relational PostgreSQL database hosted on Neon cloud.

The motivation stems from the fragmented nature of academic resources at the university level. Students typically rely on informal channels such as WhatsApp groups for past papers or faculty feedback. Red-Track-StudentHub formalises and digitises these processes, improving transparency and accessibility.

2. Problem Statement

The following challenges motivate this project:

- No centralised platform exists for students to access past examination papers organised by course and year.
- Faculty evaluation is entirely informal — no structured mechanism to rate or review teachers.
- Course information is scattered across departmental notice boards and unofficial sources.
- No mapping exists between which faculty member set which examination paper.
- Administrative management of teacher and course data lacks a user-friendly digital interface.

3. Project Objectives

- Design and implement a normalised PostgreSQL relational database covering teachers, courses, exam papers, reviews, and their relationships.
- Develop a React-based frontend enabling students to search, browse, and interact with faculty and course data.
- Implement a multi-criteria teacher rating system (Teaching Clarity, Subject Knowledge, Behaviour & Respect, Punctuality) with persistent storage.
- Provide a past-paper repository linked to courses with difficulty tagging and Google Drive PDF preview.
- Expose a RESTful Express.js API connecting the frontend to the PostgreSQL database.
- Display which examination papers each teacher has set, creating academic accountability and transparency.

4. Scope

4.1 In Scope

- Teacher directory with profile cards showing designation, department, PhD supervisor status, and live rating.

- Multi-criteria anonymous review submission with cumulative score (0-20) and average (0-5).
- Past paper listing per course with year, session, difficulty tags, and embedded PDF preview.
- Teacher-to-exam-paper mapping showing which papers each teacher set (3-table JOIN).
- Course listing with paper count computed dynamically via SQL JOIN.
- Live teacher search by name with clear/reset functionality.
- Automatic rating summary recalculation using PostgreSQL UPSERT on every review submission.

4.2 Out of Scope

- User authentication and role-based access control.
- Mobile application (web-only portal).
- Real-time notifications or messaging.
- Integration with official university ERP or LMS systems.

5. System Architecture

The project follows a three-tier client-server architecture:

Tier	Technology	Responsibility
Presentation	React.js + CSS (DM Sans)	UI rendering, state management, API calls
Business Logic	Node.js + Express.js	REST API, validation, DB query orchestration
Data	PostgreSQL on Neon (cloud)	Persistent relational storage, constraints, aggregates

6. Database Design

The database is designed in Third Normal Form (3NF). Six interrelated tables form the core schema:

Table	Primary Key	Description
teachers	id (SERIAL)	Faculty profiles: name, designation, department, email, photo, phd_supervisor
courses	id (SERIAL)	Course catalogue: name, code, credit_hours
exam_papers	id (SERIAL)	Papers linked to courses: session, year, link, difficulty
teacher_exam_papers	id (SERIAL)	Junction table mapping teachers to exam papers they set
teacher_reviews	id (SERIAL)	Student reviews: 4 criteria scores (1-5), comment, timestamp
teacher_rating_summary	teacher_id	Materialised summary: overall_avg, total_reviews — UPSERT on each review

6.1 Key Relationships

- teachers (1) to (M) teacher_reviews
- teachers (1) to (1) teacher_rating_summary
- courses (1) to (M) exam_papers
- teachers (M) to (M) exam_papers via teacher_exam_papers

7. API Endpoints

Method	Endpoint	Description
GET	/api/teachers	All teachers with live rating from summary view
POST	/api/teachers	Add a new teacher
GET	/api/courses	Courses with dynamic paper count (JOIN)
POST	/api/courses	Add a new course
GET	/api/pastpapers/:courseId	Exam papers for a specific course
POST	/api/pastpapers	Upload a new exam paper
GET	/api/teacher-reviews/:teacherId	All reviews for a teacher
POST	/api/teacher-reviews	Submit review + recalculate summary (UPSERT)
GET	/api/teacher-rating/:teacherId	Rating summary for a teacher
GET	/api/teacher-papers/:teacherId	Papers set by a teacher (3-table JOIN)

8. Technology Stack

Layer	Technology	Notes
Frontend	React.js	Functional components, useState, useEffect hooks
Styling	CSS (DM Sans font)	Custom CSS, responsive grid layout
Backend	Node.js + Express	ES Modules, RESTful API
Database	PostgreSQL (Neon)	Cloud-hosted serverless PostgreSQL
Dev Tools	VS Code, Postman	Development and API testing

9. Project Timeline

Week	Phase	Deliverable
1-2	Requirements & DB Design	ER diagram, schema definition, Neon setup
3-4	Backend Development	Express API, all endpoints, DB seeding
5-6	Frontend Core Modules	Teacher list, course list, past papers viewer
7-8	Frontend Review Module	Rating form, star widget, review display
9	Integration & Testing	End-to-end testing, bug fixes
10	Documentation & Submission	Proposal, report, demo, code cleanup

10. Expected Outcome

Upon completion, Red-Track-StudentHub will deliver a fully functional database-driven academic portal demonstrating practical DBMS concepts: relational schema design, normalisation to 3NF, complex multi-table JOINS, aggregate functions, UPSERT operations, and RESTful API integration. The system serves as a reusable template for academic portals and demonstrates the team's proficiency in full-stack development with a relational database backend.